

作业 1: GPU 与 GPU 编程

Weiming HPC Training Camp × LCP AI Infra Seminars · Session 1

Preface

此次作业为 Session 1 的配套练习，内容覆盖 CUDA Programming Guide¹ 第一、第二部分的全部章节。

共有七种题型：CONCEPT 概念题、FILL-IN 填空题、MODIFY 改造题、DEBUG 修 bug 题、EXPERIMENT 实验题、HANDS-ON 动手题、FROM-SCRATCH “从零实现”题。标 Optional 的为选做题。涉及代码编辑的题目会在标题行右侧标出相应代码文件的路径（相对 assignment01/）。FROM-SCRATCH 题会配套判测脚本。

代码在仓库 assignment01/ 目录，环境配置见其中的 README.md。

AI Policy: CLAUDE.md 或 AGENTS.md。核心原则是“AI 可以帮你理解，但不能替你实现”。

Content

Module	主题	对应阅读	代码位置
0	环境准备	README.md	cuda/m0_env/
1	为什么要用 GPU	Guide 1.1、1.2.1–1.2.2	cuda/m1_why_gpu/
2	第一个 CUDA 程序	Guide 2.1 全章、2.6	cuda/m2_first_kernel/
3	SIMT 执行	Guide 2.3.1–2.3.2、1.2.2.2	cuda/m3_simt/
4	存储空间	Guide 2.3.3–2.3.5、2.3.7、1.2.3	cuda/m4_memory/
5	计时与异步初步	Guide 2.5 引言、1.2.3.3	cuda/m5_async/
6	Tile 视角	Guide 1.2.2.3、2.4、2.2	（阅读为主）
7	TileLang 与 Triton	TileLang 文档、Triton 教程	kernels/ + tests/
8	平台与编译	Guide 1.3、2.1.1、2.7	（复用 cuda/m0_env）
Bonus	matmul	—	cuda/bonus/、kernels/

0 环境准备

先把环境跑通——编译一个 minimal 的 CUDA 程序，确认工具链和卡都能用；再把这块卡的参数查一遍，后面的 Module 会反复用到。环境配置见 README.md。

参考资料 assignment01/README.md；查硬件参数用 Guide 第五部分附录 Compute Capabilities。

prob 0.1 (HANDS-ON)

cuda/m0_env/01_hello.cu

编译并运行第一个程序，它启动 4 个 block、每个 block 8 个线程。

```
cd assignment01/cuda
make run/m0_env/01_hello
```

¹<https://docs.nvidia.com/cuda/cuda-programming-guide/>，2025 年重排的新版。下文引用记作 Guide。

看看输出的结果，建议至少运行 5 次，可以留意输出各行顺序的变化。源码可以先不细读，Module 8 会回头看 `nvcc` 对它做了什么。

prob 0.2 (FILL-IN)

`cuda/m0_env/02_device_query.cu`

`02_device_query.cu` 的五个空都是 `cudaDeviceProp` 的字段名，对照 CUDA Runtime API 文档补全，然后编译运行。

```
cd assignment01/cuda
make run/m0_env/02_device_query
```

把你这块卡的参数填进下表，并对照 Guide 附录 Compute Capabilities 里对应架构的表，核对 warp 大小和每 SM 最大线程数。

项目	你的卡
型号 / compute capability	NVIDIA GeForce RTX 5090 / 12.0
SM 数量	170
warp 大小	32
shared memory / block	48 KiB (49152 B)
最大常驻线程 / SM	1536
显存总量	约 31.36 GiB (33668857856 B, 标称 32 GB)

Editor's Note 本模块两道题看上去只是热身，但 prob 0.2 的那张表后面会反复被引用：Module 3 讨论 warp 要用 warp 大小，Module 4 手算驻留 block 数要用“shared memory / SM”与“最大常驻线程 / SM”。Keep it at hand.

另外，我们将会逐渐领悟到：一段在 GPU 上运行的代码，其性能与硬件型号是强绑定的。同一段代码在不同 compute capability 上的性能可以相差数倍，某些优化只在某几代 compute capability 上奏效。此后每记录一个耗时，请将硬件型号与 compute capability 一同记下。

1 为什么要用 GPU

GPU 与 CPU 的根本区别在设计目标——CPU 为单线程延迟服务，GPU 为总吞吐服务。此 Module 关注延迟/吞吐、SIMD/SIMT 等相关概念。

参考资料 Guide 1.1、1.2.1–1.2.2；LCPU 讲义^a Part 0。

^a<https://github.com/interestingLSY/CUDA-From-Correctness-To-Performance-Code/blob/master/lecture.md>

prob 1.1 (CONCEPT)

判断对错，可以顺带补一句理由。

(a) 一块标称 100 TFLOPS 的 GPU，执行单条指令的延迟一定低于 5 GHz 的 CPU。

回答：错。TFLOPS 衡量并行吞吐，GHz 表示时钟频率，都不能直接代表单条指令的延迟。

(b) HBM 的“高带宽”指大块连续访问时的吞吐，零散的随机访问达不到标称值。

回答：对。连续访问便于合并访存；随机访问会产生分散的内存事务，难以达到标称带宽。

(c) 严格串行的迭代算法（每步依赖上一步的结果），即使换一块算力更强的 GPU 也快不了多少。

回答：对。步骤间的依赖限制并行度，总耗时主要由串行依赖链的延迟决定。

(d) “算力 1000 TFLOPS” 意味着每次运算的延迟是 10^{-15} 秒。

回答：错。1000 TFLOPS 是整体吞吐指标，其倒数不是单次运算延迟。

prob 1.2 (CONCEPT)

Session 1 讲座里提过“N 方过百万”这个例子。总计算量 10^{12} FLOP 在当代 GPU 上的运算时间大概是毫秒级，那为什么一个严格在线的串行算法仍然做不到几秒内跑完？（从“延迟”和“吞吐”的角度考虑）

回答：GPU 依靠大量并行任务隐藏延迟并发挥吞吐；严格串行时，每一步都要等待上一步，既无法隐藏延迟，也无法利用并行算力。

prob 1.3 (CONCEPT)

补全下表（thread 一行已填好作为示例）

执行层次	软件含义	对应硬件	直接可用的存储	同步与通信手段
thread	kernel 的最小执行单位	计算单元上的一个 lane	自己的寄存器	（自身天然有序）
warp	32 个 thread 组成的 SIMT 执行组	SM 内的一组执行 lane	各 thread 的寄存器	shuffle、vote、__syncwarp 等 warp 原语
block CTA	/ 可协作的一组 thread，也是分配到 SM 的单位	整个 block 驻留在同一个 SM	shared memory 与各 thread 的寄存器	shared memory、__syncthreads、原子操作
grid	一次 kernel launch 的全部 block	整块 GPU 上的多个 SM	global memory	global memory、原子操作；通常以 kernel 边界完成全 grid 同步

prob 1.4 (CONCEPT)

SIMD 与 SIMT 的区别？另：判断正误——Nvidia GPU 在 Volta 之后每个线程有独立的 program counter，所以 branch divergence 不再有性能代价。

回答：SIMD 用一条显式向量指令处理多个元素；SIMT 让程序员编写独立标量线程，硬件再将其组成 warp 共同发射。判断为错：Volta 的独立线程调度没有改变 warp 发射方式，分支路径仍需分别执行，空闲 lane 仍会造成损失。

prob 1.5 (EXPERIMENT)

cuda/m1_why_gpu/01_scaling.cu

同一个向量加法，四种配置各跑一遍。

```
cd assignment01/cuda
make run/m1_why_gpu/01_scaling
```

将数据填入下表，并回答：(a) GPU 单线程为什么比 CPU 慢这么多？(b) 从单 block 到铺满 grid 的提速，说明 GPU 加速计算靠的是什么？

配置	耗时 (ms)	ns / 元素
CPU 单线程	16.127	3.85
GPU <<<1,1>>>	136.282	32.49
GPU <<<1,256>>>	2.338	0.56
GPU 铺满 grid	0.025	0.01

运行配置：铺满 grid 时使用 16384 个 block，每个 block 256 个 thread；结果为 PASS。

回答：

- (a) CPU 针对单线程低延迟优化；<<<1,1>>> 只有一个有效 lane，既浪费 GPU 资源，也没有其他 warp 隐藏延迟，因此更慢。
- (b) 铺满 grid 后，大量 block 分布到多个 SM，活跃 warp 可隐藏延迟并利用带宽，说明 GPU 加速依赖大规模并行吞吐。

prob 1.6 (FROM-SCRATCH·Optional)

kernels/simt_sim.py

SIMT Simulator ——写一个 warp 的执行模拟器：32 个 lane、共享控制流、带 mask 的分支执行与汇合。请在 kernels/simt_sim.py 内完成（docstring 里面有具体实现要求）。工作量大概是 50 行以内的 Python。不需要 GPU。注：本题为 Module 3 的实验结果提供理论对照。判测命令如下：

```
cd assignment01
uv run pytest tests/test_simt_sim.py
```

Editor’s Note 延迟与吞吐的分野根植于设计取舍，而非工艺差异。把“同一笔晶体管预算”投向不同的方向：一边是乱序执行、分支预测与庞大的 cache，另一边则是更多计算单元和更宽的访存带宽。prob 1.5 用四档配置让这个判断变得可度量，prob 1.1 与 prob 1.4 分别从两侧划出了它的边界，选做的 prob 1.6 则预先准备了一份理论标尺，留待 Module 3 的实测去校准。

更值得记住的是 prob 1.2 得出的那条结论：无论可用吞吐有多高，它始终绕不过一条串行依赖链。判断某个任务是否值得搬上 GPU，最先看的不是总计算量，而是依赖“图”的宽度——Amdahl 定律在 GPU 场景下的变体大抵如此。至于 prob 1.5 的那份向量加法，它几乎榨不出这块卡真正的算力，真正的瓶颈藏在另一个地方；Module 4 会为你把它量化出来。

2 第一个 CUDA 程序

此 Module 把一个 CUDA 程序的完整生命周期过一遍，覆盖 Guide 2.1 的每个小节。

参考资料 Guide 2.1 全章（主要出处）、2.6（浏览 unified memory 相关小节）；LCPU 讲义 Part 1。

prob 2.1 (FILL-IN)

cuda/m2_first_kernel/01_vector_add.cu

请填空：m2_first_kernel/01_vector_add.cu，具体要求见代码注释。判测命令如下：

```
cd assignment01/cuda
make run/m2_first_kernel/01_vector_add
```

prob 2.2 (CONCEPT)

为下列五个场景选择正确的修饰符（如 `__global__` 等）。

- (a) 在 GPU 上执行、由 CPU 侧启动的 kernel 函数。

回答: `__global__`

- (b) 只会被 kernel 调用的辅助函数。

回答: `__device__`

- (c) host 和 device 代码都要调用的小工具函数。

回答: `__host__ __device__`

- (d) 整个 kernel 运行期间不变、所有线程都要读的系数表。

回答: `__constant__`

- (e) block 内线程共享的暂存数组。

回答: `__shared__`

prob 2.3 (MODIFY)

`cuda/m2_first_kernel/02_vector_add_um.cu`

`02_vector_add_um.cu` 代码完整，但目前是“显式内存管理”的版本。改之前先按原样编译运行一次，记下耗时——这一版会被你的改动覆盖掉，下面 (b) 要拿它做对照。然后按文件头的说明改成 unified memory 版（`cudaMallocManaged`），并保持文件头写明的计时窗口不变。

```
cd assignment01/cuda
make run/m2_first_kernel/02_vector_add_um
```

然后请回答如下问题：(a) kernel 启动之后、CPU 读结果之前，为什么必须有一次同步？在原先的版本里这次同步发生在哪个调用里？(b) 对比两版“搬运 + kernel + 读回”的耗时，分析差距的原因（谁快谁慢都有可能，与使用的卡有关）。

回答：(a) kernel 异步执行，CPU 读结果前必须等待 GPU 写完。当前版本显式同步；原版本的 device-to-host `cudaMemcpy` 也会等待此前的 kernel。

(b) 显式内存版为 110.9 ms，Unified Memory 版为 94.9 ms，后者快约 1.17 倍。本机的按需页面迁移开销较低，但结果依赖硬件互连和迁移策略。

prob 2.4 (CONCEPT)

判断对错，可以顺带补一句理由。

- (a) `vectorAdd<<<...>>>(...)` 这条语句返回时，kernel 一定已经执行完毕。

回答：错。kernel 启动通常是异步的，返回只表示任务已提交。

- (b) 同一个 stream 里，`cudaMemcpy`（device 到 host）会等它前面的 kernel 全部完成后才开始拷贝。

回答：对。同一 stream 按顺序执行，同步拷贝会等待此前的 kernel。

- (c) kernel 内部的非法访存，会在启动语句处同步地报出来。

回答：错。启动处只能立即发现配置错误；非法访存等异步错误要到同步或后续 CUDA API 调用时才会报告。

prob 2.5 (DEBUG)

`cuda/m2_first_kernel/03_bug_launch.cu`

修 bug: `03_bug_launch.cu`，详细内容见相关文件。

```
cd assignment01/cuda
```

```
make run/m2_first_kernel/03_bug_launch
```

回答：每个 block 的 2048 个线程超过 `maxThreadsPerBlock`，导致 `invalid configuration argument`。加入启动错误检查并将线程数改为 1024 后通过；该上限针对单个 block。

prob 2.6 (FILL-IN)

cuda/m2_first_kernel/04_matrix_add.cu

04_matrix_add.cu 用二维的 block 和 grid 处理 1000×700 的矩阵，请填空实现矩阵加法。测试命令：

```
cd assignment01/cuda
make run/m2_first_kernel/04_matrix_add
```

prob 2.7 (MODIFY)

cuda/m2_first_kernel/05_grid_stride.cu

05_grid_stride.cu 的 launch 被固定成 `<<<64, 256>>>`，线程总数远小于 n ，当前 FAIL。在 launch 配置不变的前提下，把 kernel 改成 grid-stride loop，让任意 n 都能 PASS。

```
cd assignment01/cuda
make run/m2_first_kernel/05_grid_stride
```

然后请回答——这种写法的价值在哪里？launch 只有 16384 个线程时，性能上要付出什么代价？

回答：Grid-stride loop 用固定线程数处理任意规模数据，并保持合并访存。代价是循环开销；block 太少时还会降低并行度和延迟隐藏能力。

prob 2.8 (EXPERIMENT)

cuda/m2_first_kernel/06_whoami.cu

运行下面的程序两三次，观察 16 个 block 打印输出的先后。

```
cd assignment01/cuda
make run/m2_first_kernel/06_whoami
```

(a) 顺序由谁决定？(b) 程序的正确性可以依赖 block 的执行顺序吗？这条限制和 Guide 1.1 说的 `scalable programming model` 有什么关系？

回答：(a) 顺序由 GPU 根据 SM 和资源状态调度，不保证按 `blockIdx.x` 排列。

(b) 正确性不能依赖 block 顺序；block 相互独立，才能被任意分配和分批执行，从而适配不同规模的 GPU。

prob 2.9 (FROM-SCRATCH): SAXPY

cuda/m2_first_kernel/saxpy.cu

在 `m2_first_kernel/` 下写出完整的 CUDA 程序 `saxpy.cu`，实现 $y \leftarrow 2.0 \cdot x + y$ （单精度）。要求如下：

- 不允许 include `common.h`。错误检查宏和 `cudaEvent` 计时都要自己写一遍。
- 命令行用法：`./saxpy <n>`， n 是元素个数。输入数据按固定公式生成（都是 float）： $x[i] = ((i \% 2048) - 1024) * 0.5f$ ， $y[i] = (i \% 1024) - 512$ 。
- kernel 算完把 y 拷回 host，用 double 累加所有 $y[i]$ ，输出一行 `SUM=< 总和 >`（用 `printf("SUM=%.0f\n", s)` 这样的格式，同一行里可以再带上 n 和 kernel 毫秒数），SUM 结果将用于对拍检验程序正确性，exit code 应为 0。
- $n = 0$ 时输出 `SUM=0`，exit code 为 0（0 个 block 的 kernel launch 是非法的，特判即可）。

判测脚本覆盖 $n \in \{0, 1, 31, 1024, 1025, 2^{20}, 2^{20}+3\}$ ，命令如下。

```
cd assignment01/cuda/m2_first_kernel
./judge_saxpy.sh saxpy.cu
```

注：SAXPY 即 Single-precision A·X Plus Y。

Editor's Note CUDA C++ 在 C++ 之上添加的语法扩展其实相当克制：几种函数与变量修饰符、一个 kernel 启动语法，以及一组内存管理 API。真正需要你花时间去适应的是另外两件事——host 与 device 各自独立推进的时间线（prob 2.4），以及数据此刻落在谁的地址空间里，你得“显式”地负责（prob 2.3）。作为本模块收尾的 prob 2.9 把这些概念连同索引、边界检查与错误处理一起收进一个不到百行的程序，prob 5.1 还会回过头来检验其中的计时部分。

prob 2.7 和 prob 2.8 值得多花一些时间。你为这两道题写下的理由——一条关于启动配置与问题规模的关系，另一条关于 block 之间的执行顺序——在后面的内容里会以不同面貌反复出现。Module 7 介绍的 Triton 与 TileLang 仍然建立在同一条 block 无序执行的假设之上，只不过不再需要你亲手把它写出来。

3 SIMT 执行

SIMT 给每个线程“独立执行”的表象，硬件却按 32 线程一组共享 instruction fetch 和 issue（取指和发射）。此模块三个实验的主题分别为“divergence 的代价”、“__syncthreads 的必要性”、“block 之间为什么无法同步”。

参考资料 Guide 2.3.1–2.3.2、1.2.2.2。3.3、3.5 要用 shared memory，用法见 Guide 2.3.3.2 与 1.2.3.2（Module 4 会系统地读这一块）；cooperative groups 见 Guide 2.3.6（只需浏览）。

prob 3.1 (CONCEPT)

设 `blockDim = (8, 8, 1)`。

(a) `threadIdx = (3, 5, 0)` 的线性编号是多少？它在第几个 warp、warp 内第几个 lane？

回答：线性编号为 $3 + 5 \times 8 = 43$ ；warp 编号为 $\lfloor 43/32 \rfloor = 1$ （第 2 个），lane 编号为 $43 \bmod 32 = 11$ （第 12 个）。

(b) 这个 block 一共占多少个 warp？

回答： $8 \times 8 = 64$ 个线程，恰好占 2 个 warp。

(c) 若 `blockDim = (33, 1, 1)`，占几个 warp？这样配置浪费在哪里？

回答：占 2 个 warp；第二个 warp 仅 1 个 lane 有效，浪费 31 个执行槽位。

prob 3.2 (EXPERIMENT)

`cuda/m3_simt/01_divergence.cu`

`m3_simt/01_divergence.cu` 的两个 kernel 每线程计算量相同，分支划分不同——一个按 thread 编号的奇偶分（同一个 warp 里一半一半），一个按 warp 边界对齐分。请先预测一下哪个版本运行会更快一点，大概快几倍，然后运行验证：

预测：按 warp 分支约快 2 倍；奇偶分支使每个 warp 串行执行两条路径，而按 warp 分支不分散。

```
cd assignment01/cuda
make run/m3_simt/01_divergence
```

请解释实测比值，并回答——若两个分支的计算量一大一小，按 thread 编号奇偶分的 kernel 和按 warp 边界对齐分的 kernel 的运行时间分别由什么决定？

回答：实测为 0.387 ms 和 0.200 ms，比值 1.94，符合预测；偏离 2 倍来自共同开销和测量波动。若两条路径计算量不同，奇偶分支的单 warp 耗时近似为两者之和；按 warp 分支时，每个 warp 只承担所选路径，grid 耗时由两类 warp 的总工作量和较重路径的收尾决定。

prob 3.3 (EXPERIMENT)

cuda/m3_simt/02_sync_matters.cu

02_sync_matters.cu 让每个 block 用 shared memory 把自己的 256 个元素倒序。请按文件开头的注释内容进行实验。

```
cd assignment01/cuda
make run/m3_simt/02_sync_matters
```

(a) 为什么注释掉 sync 后代码不能正确地运行？

回答：线程读取的是其他线程写入的 buf[255-t]。没有 __syncthreads() 时可能先读后写，产生竞争；该同步保证所有写入完成并可见。

(b) (Optional) 注释掉 sync 后，翻转后的数组错的位置比较随机，但是有些位置一直是对的，试解释原因。(tip: 算一算 t 与 $255 - t$ 有没有可能落在同一个 warp)

回答： t 在 warp w ， $255 - t$ 在 warp $7 - w$ ； $w = 7 - w$ 无整数解，因此二者总在不同 warp。部分位置持续正确只是实际调度顺序较稳定的偶然结果，CUDA 不作保证。

prob 3.4 (CONCEPT)

__syncthreads 只能同步本 block 内的 threads，那需要全 grid 同步时，标准做法是什么？

回答：把计算拆成同一 stream 中的多个 kernel；后一个 kernel 会等待前一个全部完成，因此 kernel 边界就是全 grid 同步，阶段间通过 global memory 传递数据。

prob 3.5 (FROM-SCRATCH): block 内归约

cuda/m3_simt/03_reduce.cu

在 03_reduce.cu 里从零实现两个求和归约 kernel (判测与计时的代码已经写好)。两个 kernel 的 contract 见文件头。PASS 后，试解释实测性能差距的原因。

(Optional) 基于两点事实——(a) __shfl_down_sync 是 warp 内寄存器级别的线程间数据交换指令，自带同步效果且延迟极小；(b) 归约到最后 32 个元素后，活跃线程若都落在同一个 warp 里，就不再需要 __syncthreads (可以想想为什么)——据此试写出第三版优化后的 kernel。测试时只会跑前两版，第三版自己在 main 里照着加一次 run_one 调用即可。

```
cd assignment01/cuda
make run/m3_simt/03_reduce
```

结果与回答：关闭 device 优化后，三版均 PASS: interleaved 为 0.0369 ms，contiguous 和 shuffle 均为 0.0246 ms。交错版因取模、活跃 lane 分散和分支发散而慢 1.50×；本次测试未测出 shuffle 的额外加速。

Editor's Note SIMT 让你以单线程的风格编写并行代码，但它只承诺正确性语义，不负责性能语义。本模块的题目正好分别对应这层抽象漏出来的三个口子：warp 才是真实的调度单位（prob 3.2），block 内的并行需要显式同步才能看到彼此的写入（prob 3.3），而 block 之间压根没有提供同步方法（prob 3.4）。

“For performance reasoning, you always need to fall back to the warp level.”

prob 3.5 抛出的结论更让人不适：两个版本的加法次数完全相同，耗时却能相差超过一倍。算法复杂度在这里不再是性能的充分描述——这大概是 GPU 编程与经典算法课之间最根本的分歧。选做的第三版则指向另一条路径：不再把 warp 当作需要小心绕开的硬件细节，而是直接把它当作可调用的编程接口。warp-level primitive 与 cooperative groups（Guide 2.3.6）正是这个方向的官方解答。

4 存储空间

数据的存储位置和迁移速度极大地影响着 kernel 的速度。此 Module 旨在对“存储如何影响性能”建立起基础认知。

参考资料 Guide 2.3.3–2.3.5、2.3.7、1.2.3。

prob 4.1 (CONCEPT)

补全下表：

空间	谁可见	生命周期	片上 / 片外	谁管理
register	单个线程	线程	片上	编译器
local	单个线程	线程	片外（经缓存）	编译器
shared	同一 block	block	片上	程序员
global	所有线程；host 经 API	分配至释放	片外	程序员 / runtime
constant	所有线程只读；host 写	程序 / module	片外（片上 cache）	程序员 / runtime
L1 / L2 cache	L1: 同一 SM; L2: 全 GPU	缓存驻留期间	片上	硬件

prob 4.2 (FILL-IN)

cuda/m4_memory/01_stencil.cu

请填空：m4_memory/01_stencil.cu，具体要求见代码注释。测试命令：

```
cd assignment01/cuda
make run/m4_memory/01_stencil
```

prob 4.3 (MODIFY)

cuda/m4_memory/02_constant_coeff.cu

02_constant_coeff.cu：按文件头的说明进行修改。修改完后测试：

```
cd assignment01/cuda
make run/m4_memory/02_constant_coeff
```

结果会包含两个版本的耗时（差距可能极小，甚至测不出来性能收益——想想为什么），回答 constant cache 真正的优势在哪种访问模式。

回答：两版均 PASS，耗时均为 0.0718 ms，比值 1.00×。constant cache 的优势是同一 warp 读取同一地址时可广播数据；本题仅有 8 个系数，global 版本也容易命中缓存，因此未测出收益。

prob 4.4 (CONCEPT)

判断对错，可以顺带补一句理由。

(a) local memory 的“local”指作用域私有，它实际上在片外显存里。

回答：对。local 表示每个线程私有，物理上位于片外显存，并可经过 L1/L2 cache。

(b) 对数组用运行期才知道的下标做索引，可能迫使它被放进 local memory。

回答：对。寄存器不能动态寻址，编译器可能把这类数组放入可寻址的 local memory。

prob 4.5 (FILL-IN)

cuda/m4_memory/03_histogram.cu

请填空：03_histogram.cu，具体要求见代码注释。测试命令：

```
cd assignment01/cuda
make run/m4_memory/03_histogram
```

prob 4.6 (MODIFY)

cuda/m4_memory/04_histogram_priv.cu

04_histogram_priv.cu：请按要求修改代码。改完测试指令：

```
cd assignment01/cuda
make run/m4_memory/04_histogram_priv
```

测试结果包含与 naive 实现相比较的耗时、吞吐。试解释提速来自哪里。

回答：naive 为 2.6092 ms，私有化版为 0.0754 ms，提速 34.60×。私有化把大部分竞争转移到低延迟的 block-local shared memory，最后每个 block 仅合并 256 个 bucket，大幅减少了 global atomic 操作和跨 block 竞争。

prob 4.7 (EXPERIMENT)

cuda/m4_memory/05_bandwidth.cu

运行实验，并根据实验数据填表。

```
cd assignment01/cuda
make run/m4_memory/05_bandwidth
```

观察数据变化趋势，并简析趋势的成因。

stride	1	2	4	8	16	32
GB/s	1883.9	1196.1	738.7	596.4	1619.7	860.1

回答：stride 从 1 增至 8 时，warp 的访问逐渐分散，需要更多内存事务，带宽下降。由于 n 和 stride 都是 2 的幂，取模后实际只访问 n/stride 个不同元素；stride 继续增大时，工作集缩小、

缓存复用增强，并受 cache set 与 memory partition 映射影响，因而出现非单调波动。表中数值按逻辑读写量计算，不等同于实际 DRAM 流量。

prob 4.8 (EXPERIMENT)

cuda/m4_memory/06_occupancy.cu

06_occupancy.cu，详细内容见相关文件。实验命令如下：

```
cd assignment01/cuda
make run/m4_memory/06_occupancy
```

记录实验数据：

shared memory / block (KB)	0.0	13.2	15.0	18.0	29.0	55.0
理论驻留 block / SM	6	6	6	5	3	1
occupancy	100.0%	100.0%	100.0%	83.3%	50.0%	16.7%
实测带宽 (GB/s)	1572.9	1572.2	1574.8	1576.5	1579.5	820.4

请回答：(a) 用程序开头打印的“shared memory / SM”和“最大常驻线程 / SM”，手算其中一个的驻留 block 数和 occupancy，和 API 的结果对照。(b) 带宽为什么随 occupancy 下降？用“延迟隐藏需要足够多的常驻 warp”组织你的解释。(c) 表中带宽随 occupancy 单调下降，但明显不成正比——从 100% 到 75% 带宽掉了多少？从 37.5% 到 12.5% 又掉了多少？试解释这个差别。

回答：(a) 以 29.0 KB 为例，每个 SM 最多驻留 $\lfloor 100/29 \rfloor = 3$ 个 block，occupancy 为 $3 \times 256/1536 = 50\%$ ，与 API 一致。

(b) occupancy 降低会减少常驻 warp；当可调度的 warp 不足时，访存等待无法被其他 warp 的执行隐藏，带宽便会下降。

(c) 本机最大常驻线程数为 1536，故没有题面预设的 75%、37.5% 和 12.5% 档位。实测从 100% 降至 50% 时带宽基本不变，说明 24 个常驻 warp 已足以隐藏延迟；从 50% 降至 16.7% 时，带宽由 1579.5 降至 820.4 GB/s，下降约 48.1%，因为仅剩 8 个常驻 warp，延迟隐藏能力不足。

Editor's Note 本模块的八道题围绕同一个问题展开：数据放在哪一层，以什么模式去取。prob 4.1 给出一幅静态的存储层次地图，prob 4.2 与 4.3 在同一段 stencil 上切换存储位置，prob 4.5 与 4.6 在同一个直方图统计上做同样的变换，prob 4.7 和 4.8 则分别量化访问模式与可并发的请求数量。

有三点值得单独记下。其一，优化只在真正的瓶颈处生效。prob 4.3 是一个刻意安排的负结果：教科书罗列的手段用在了不构成瓶颈的地方，收益为零。避免这类空转，需要先估算再动手，估算可以依赖 arithmetic intensity 与 roofline 模型，后面的 session 会展开。其二，你在 prob 4.8 (b) 写下的那句解释有一个正式的名称——Little 定律：维持某一吞吐量所需的在途请求数，等于吞吐与延迟的乘积。其三，occupancy 本身是手段，不是目标。它高只说明调度器有足够多的 warp 可以切换，并不自动保证访存效率；反过来，低 occupancy 配合足够的指令级并行，同样能跑满内存带宽。Volkov 的 *Better Performance at Lower Occupancy* (GTC 2010) 是这一观点的经典论述。

5 计时与异步初步

在前面的练习里 `cudaDeviceSynchronize` 反复出现以保证计时的正确性。本 Module 主要关注两点——kernel 启动是异步的；以及如何准确计时。

参考资料 Guide 2.5（只读引言）、1.2.3.3。

prob 5.1 (EXPERIMENT)

cuda/m5_async/01_timing_trap.cu

运行实验：

```
cd assignment01/cuda
make run/m5_async/01_timing_trap
```

并回答下列问题：(a) 哪个数值可以当作 kernel 耗时写进报告？(b) 另外两个各具体测的是什么？

回答：(a) `cudaEvent` 测得的 0.0665 ms 可作为 kernel 耗时。

(b) 不等 GPU 的 0.0065 ms 只测到 host 提交 kernel 的开销；等 GPU 的 0.0691 ms 测的是从提交到 GPU 完成的 host 墙钟时间，还包含启动、同步及等待开销。

prob 5.2 (CONCEPT)

判断对错，可以顺带补一句理由。

(a) 同一个 stream 里的操作按提交顺序执行。

回答：正确。同一 stream 内的操作按入队顺序执行。

(b) kernel 启动后，host 代码立刻继续往下执行。

回答：正确。kernel launch 通常对 host 异步，除非随后显式同步。

(c) unified memory 下，CPU 访问一页正被 GPU 占用的内存，会触发缺页与页迁移。

回答：正确。CPU 缺页后，运行时会协调访问并将该页迁移到 CPU。

Editor's Note Module 5 的两道题分量不算重，但在整个认知链条里恰好落在一个关键的位置。CUDA 里的异步并不是某种需要额外开启的特性，它更像是 kernel launch 的默认语义：你把工作提交到某条 stream 上，host 端立刻返回，device 端的执行则在另一条时间线上独立展开。正因如此，“计时”这件事本身变成了一门需要认真对待的事——warmup、同步点插在哪里、选哪种 timer、重复多少次，任何一环处理得马虎，拿到的数字就失去了意义。这次作业里，这些细节已经由写好的测试框架替你包揽了；等到将来你自己搭建 benchmark，就得把这些一一补回来。

再下一步，异步自然会引出重叠：多 stream、数据拷贝与计算的并发、用 CUDA Graph 把整张任务图一次性提交。这些内容会放在后续 session 里展开，这里只需要建立起一个清晰的 mental model：host 只管提交，device 只管执行，两条时间线各自推进。

6 Tile 视角

Guide 从 13.x 起把 tile 编程作为与 SIMT 并列的第二种官方模型写进正文。tile 模型描述的是“一个 block 对一块数据做什么”，而 block 内 threads 的分工由编译器决定。此 Module 只做

概念铺垫和对照阅读。

参考资料 Guide 1.2.2.3（必读，篇幅不长）、2.4.1–2.4.6（浏览）、2.2（知道 CUDA Python 这条路线存在即可）。

prob 6.1 (CONCEPT)

判断对错，可以顺带补一句理由。

- (a) tile 是显存里的一块可变区域，kernel 通过指针直接改写它。
- (b) 对 tile 的一次运算（如两个 tile 相加）由编译器映射到 block 内的多个线程上执行。
- (c) tile 模型与 SIMT 模型互斥，一个 CUDA 程序只能选一种。

prob 6.2 (CONCEPT)

下面是 Guide 2.4.6 的 cuTile Python 向量加法：

```
import cuda.tile as ct

@ct.kernel
def vec_add(a, b, c, TILE: ct.Constant[int]):
    a_view = a.tiled_view((TILE,))
    b_view = b.tiled_view((TILE,))
    c_view = c.tiled_view((TILE,))

    bid = ct.bid(0)
    a_tile = a_view.load((bid,))
    b_tile = b_view.load((bid,))
    c_view.store((bid,), a_tile + b_tile)
```

据此补全下表（Triton 一列可以做完 Module 7 再回来填）：

	CUDA SIMT	cuTile	Triton
并行单位	block 里的 thread	block	
编号	blockIdx threadIdx	/	
数据分工	线程用全局下标来 划分数据		
边界处理	if 判断		

prob 6.3 (CONCEPT)

仍看上面这段代码。(a) “每个线程对应哪个/些元素”由谁决定？(b) 列出一些在 CUDA SIMT 版向量加法里一定会出现、这里完全没体现出的概念。

Editor’s Note tile 并非某个 DSL 的发明。NVIDIA 自己把它与 SIMT 并列写进了 Guide 正文，这本身就说明它是一类稳定的编程方式，而不是一次工具选型。

从 SIMT 转向 tile，真正改变的是抽象层的职责划分：线程到数据的映射交给了编译器，tile

的尺寸却仍由用户来定。这条界线不是随意画下的——前者凭形状就可以机械推导，后者必须依赖硬件参数与实测，编译器暂时还无力接手。prob 7.5 的表格会把这一判据推广到四种模型上。

需要明确的是，抽象层的上移并不会抹掉底层的硬件。coalescing、shared memory 的容量、occupancy 这些约束仍然在起作用，只不过不再由你亲手去处理。这也是本作业先用五个模块讲清楚 SIMT，再引入 DSL 的缘故。

7 TileLang 与 Triton

此 Module 用 Triton / TileLang 重写一部分之前用 CUDA SIMT 实现的 kernel，重心在 TileLang。注：Triton 题（7.1、7.2、7.8）不需要 GPU 也能完成正确性部分，运行方式见 README.md；TileLang 题都需要 GPU，在集群上跑。

参考资料 TileLang Language Basics^a（建议阅读）；Triton 官方教程 01-vector-add^b；tilelang-puzzles^c。

^ahttps://tilelang.com/programming_guides/language_basics.html

^b<https://triton-lang.org/main/getting-started/tutorials/>

^c<https://github.com/tile-ai/tilelang-puzzles>

prob 7.1 (FILL-IN)

kernels/vector_add.py

请填空：kernels/vector_add.py，具体要求见代码注释。测试命令：

```
cd assignment01
uv run pytest tests/test_vector_add.py
```

prob 7.2 (MODIFY)

kernels/fused_op.py

按文件开头注释要求修改：kernels/fused_op.py。测试命令：

```
cd assignment01
uv run pytest tests/test_fused_op.py
```

改完回答——与 Module 2 里改 CUDA kernel 相比，这次的改动主要集中在 kernel 的什么部分？主体代码为什么一行都不用动？

prob 7.3 (FILL-IN)

kernels/tilelang_scale_add.py

请填空：kernels/tilelang_scale_add.py，具体要求见代码注释。测试命令：

```
cd assignment01
uv sync --extra tilelang
uv run pytest tests/test_tilelang.py -k scale_add
```

prob 7.4 (FILL-IN)

kernels/tilelang_copy2d.py

请填空：kernels/tilelang_copy2d.py，具体要求见代码注释。测试命令：


```
cd assignment01
uv run pytest tests/test_tilelang.py -k copy2d
```

填完想一想：2.6 的四个空——行号、列号、边界保护、grid 尺寸——哪些在这里还有对应？没有对应的那个去哪了？

prob 7.5 (CONCEPT)

补全下表，每个空填“用户”或“编译器”（二者都涉及的要写清楚各自的范围）。

谁负责	CUDA SIMT	cuTile	Triton	TileLang
线程到数据的映射	用户			
边界处理	用户			
tile / block 尺寸的选择	用户			
block 内同步	用户			

prob 7.6 (FILL-IN)

kernels/tilelang_matmul.py

请填空：kernels/tilelang_matmul.py，具体要求见代码注释。测试命令：

```
cd assignment01
uv run pytest tests/test_tilelang.py -k matmul
```

填完回答——这五个空涉及到了 shared memory、寄存器 tile、流水，而 Triton 版 matmul (Bonus 的 kernels/matmul_triton.py) 并未被显式指定，为什么？

prob 7.7 (FROM-SCRATCH): softmax in TileLang

kernels/tilelang_softmax.py

在 kernels/tilelang_softmax.py 里从零实现行 softmax，具体要求见文件开头的 docstring。

```
cd assignment01
uv run pytest tests/test_tilelang_softmax.py
```

prob 7.8 (FROM-SCRATCH·Optional)

kernels/softmax.py

用 Triton 重写 prob 7.7，此题可以不用 GPU。写完后可以试试对比一下此题与 7.7：归约、边界处理、按形状编译，二者各自是由谁处理的（用户显式处理或者编译器隐式处理）？

测试命令：

```
cd assignment01
uv run pytest tests/test_softmax.py
```

Editor's Note 本模块把前面写过的 kernel 用 TileLang 和 Triton 各重写了一遍。两种写法的区别，基本都收在 prob 7.5 那张对比表里：线程到数据的映射归谁管，边界归谁处理，tile 尺寸归谁选定，同步又归谁插入。DSL 的价值并不在于代码更短，而在于它把前两项交给编译器，同时把后两项——那些必须靠实测才能定下来的旋钮——继续留给你。

抽象当然有代价。Bonus 部分的一组数字提醒你，表达能力和实现成熟度需要分开评估：同

一段 TileLang 换个版本，跑出的结果就可能不一样。更关键的是，一旦编译器接手的那部分出了问题——layout 不合法、精度对不上、边界被悄悄吃掉——你的排查路径最终还是得退回 SIMT 这一层，去看它究竟生成了什么。DSL 是构筑在 CUDA 之上的一层，从来不是它的替代品。

8 平台与编译

参考资料 Guide 1.3 全章（必读）、2.1.1、2.7（浏览）。

prob 8.1 (CONCEPT)

判断对错，可以顺带补一句理由。

- (a) PTX 是 GPU 直接执行的机器码。
- (b) 只嵌入了 `sm_70` SASS 的可执行文件，能在 compute capability 9.0 的卡上运行。
- (c) 一个 fatbin 可以同时携带多个架构的 SASS 和 PTX。
- (d) JIT 编译由驱动在运行时完成。

prob 8.2 (EXPERIMENT·Optional)

`cuda/m0_env/01_hello.cu`

两个编译实验，对象是 Module 0 的 `01_hello.cu`。

(a) 生成只含 `sm_90` SASS 的可执行文件并运行（如果你的卡本身就是 compute capability 9.0，先把 `ARCH_HIGH` 调成 100 或更高再 `make`），记录报错信息。

```
cd assignment01/cuda
make sassonly/m0_env/01_hello
./bin/m0_env/01_hello_sassonly
```

(b) 生成只含 `compute_75` PTX 的版本（CUDA 13 起 `compute_70` 已被移除，Makefile 默认取 75）并运行。能正常运行吗？PTX 是在什么时候、由谁编译成这块卡的机器码的？

```
cd assignment01/cuda
make ptxonly/m0_env/01_hello
./bin/m0_env/01_hello_ptxonly
```

prob 8.3 (CONCEPT·Optional)

请简单说明 Runtime API 与 Driver API 各自的定位。`cudaMalloc` 属于哪个？

Editor's Note Module 0 里那个“能跑就行”的 hello，现在被拆开重新看过：源码经 `nvcc` 分成 host 和 device 两路，device 端产出 PTX 与 SASS，一并打包进 fatbin；运行时 driver 再从中挑选合适的版本，或者对 PTX 做 JIT。

这一模块的实用意义在交付环节。“编译通过”和“能在目标卡上跑起来”是两回事，后者是部署中最常撞上的墙。当你把一个 GPU 程序交付给别人，至少需要说清楚：它支持哪些 compute capability，fatbin 里包含了哪些架构的 SASS，有没有 PTX 兜底，以及首次运行的 JIT 开销落在哪里。写 kernel 与发布软件是两种视角，而这里正是二者的交界。

Bonus (EXPERIMENT · Optional): matmul

调参并比较不同 matmul 实现的性能差距。

- (a) Naive CUDA: `cuda/bonus/matmul.cu`, 用 `-DBS` 取 8、16、32 各跑一遍, 记录实测数据。

```
cd assignment01/cuda
make run/bonus/matmul
nvcc -O2 -std=c++17 -I. -arch=native -DBS=8 -o bin/bonus/matmul_bs8 bonus/matmul.cu
&& ./bin/bonus/matmul_bs8
```

- (b) Tiled Triton: `kernels/matmul_triton.py` (多组 tile 配置 + cuBLAS 对照), 记录实测数据。

```
cd assignment01
uv run python -c "from kernels.matmul_triton import bench; bench()"
```

- (c) TileLang: `kernels/tilelang_matmul.py` (即 prob 7.6 补全的成品), 调 `bench()` 的 `block_M / block_N / block_K / num_stages`, 记录实测数据。

```
cd assignment01
uv sync --extra tilelang
uv run python -c "from kernels.tilelang_matmul import bench; bench()"
```

试分析性能差距的原因, 特别是: TileLang 的控制粒度比 Triton 更细, 为什么在这组测试里反而略慢?

注: A100 实测参考——naive CUDA 约 2.4 TFLOPS, Triton 约 125 TFLOPS, TileLang 约 118 TFLOPS, cuBLAS 约 173 TFLOPS, 而 A100 fp16 tensor core 的标称上限是 312 TFLOPS。Triton 与 TileLang 的数字只是各自 `bench()` 里那几组配置中的最优, 不代表工具的性能上限; tilelang 本身也还在快速迭代 (本仓库固定在 0.1.12), 数字会随版本变化。

Editor's Note 回顾这份作业, 主线可以看作三次视角的切换: 单线程到 warp (Module 1–3)、计算到数据搬运 (Module 4–5)、线程到 tile (Module 6–7)。走完这一遍, 你写下的 kernel 应当能保证正确, 且不至于慢得离谱。但距离“快”字仍然有一段差距, Bonus 里那组数字恰好把它量化了出来——naive 实现和 cuBLAS 之间, 隔着 tensor core、asynchronous copy、software pipelining 以及 profiler-driven tuning, 这些内容会在后续的 session 中展开。